

AOIA Environment Audit

Date: 2026-05-25

Repository: /home/1/Desktop/AOIA-Core

Git baseline before this session: 55ed58b AOIA Phase 1: add candidate promotion triage pipeline

Scope: controlled AOIA/AIOA Core development resumption only. LSC and MHLM repositories were not modified.

Executive State

AOIA Core is a Python-first runtime with deterministic retrieval, provenance-aware memory separation policy, provider routing, local command execution, browser tooling, and a small local HTTP web adapter.

The repo is not yet organized into the requested production skeleton (**backend/**, **frontend/**, **tui/**, etc.). The current implementation is concentrated under **runtime/**, with supporting **docs/**, **tests/**, **reports/**, **provenance/**, and **memory/** folders. Restructuring should be deferred until a thin compatibility plan exists because runtime imports currently assume `PYTHONPATH=runtime`.

Repository Structure

Current active implementation areas:

Area	Status
<code>runtime/main.py</code>	Primary CLI/runtime loop
<code>runtime/webapp.py</code>	Local HTTP JSON API and static UI server, not FastAPI
<code>runtime/tools/</code>	Execution, browser, filesystem, shell, memory helpers
<code>runtime/providers/</code>	Gemini/OpenRouter/DeepSeek/Aureon provider adapters
<code>runtime/retrieval/</code>	Linux/RHCSA retrieval facade and deterministic engine
<code>runtime/knowledge/</code>	Canonical/candidate Linux knowledge stores
<code>runtime/memory/</code>	RHCSA context and worker memory
<code>tests/</code>	112 unittest tests now passing
<code>docs/</code> and <code>runtime/reports/</code>	Architecture, stabilization, and phase reports

Missing or not yet formalized:

- backend/
- frontend/
- tui/
- adapters/
- configs/
- docker/

These should not be created blindly. The safe next step is to add compatibility wrappers or a documented migration plan rather than move runtime modules immediately.

Python Environment

Python:

- System Python: 3.12.3
- Runtime venv: runtime/.venv
- Runtime venv Python: 3.12.3

Installed runtime-relevant packages after repair:

Package	Status
google-genai	installed
playwright	installed
requests	installed
websockets	installed
beautifulsoup4	installed during this session
fastapi	missing, not used by current runtime
rich	missing, not used by current runtime
textual	missing, not used by current runtime

Dependency repair:

- Added beautifulsoup4>=4.12.0 to runtime/requirements.txt.
- Installed beautifulsoup4==4.14.3 and soupsieve==2.8.4 into runtime/.venv.

Reason:

- runtime/tools/web_reader.py imports bs4.BeautifulSoup; before repair, bs4 was missing.

Node Environment

Node/NPM are available:

- Node: v22.22.2
- NPM: 10.9.7

There is no active `package.json`, lockfile, or Node build pipeline in this repo. The current web surface is static/local and served by Python `http.server` in `runtime/webapp.py`.

Backend/API State

Current backend style:

- `runtime/main.py`: terminal-oriented runtime loop.
- `runtime/webapp.py`: `ThreadingHTTPServer` JSON API with `/api/status`, `/api/models`, `/api/chat`, `/api/model`.

FastAPI status:

- Not installed.
- Not currently used.
- No FastAPI route layer exists yet.

Websocket status:

- `websockets` is installed as a transitive/available dependency.
- No operational websocket server layer is currently implemented.

Recommendation:

- Do not add FastAPI/websocket architecture until terminal prototype contracts are stabilized.
- If API work starts, wrap existing runtime through a thin adapter rather than moving runtime logic.

Terminal/UX State

Current terminal support:

- Slash commands exist through `runtime/commands/`.
- Plain `help` now routes to `/help` without model usage.
- Safe local routes can execute without interactive approval.
- Risky shell actions still request confirmation.
- Provider switching works through `/model`.
- Runtime status snapshot works.

Not yet present:

- Textual TUI.
- Rich dashboard rendering.
- Live terminal dashboard.
- Streaming log pane.

Recommendation:

- Build a minimal TUI shell around the existing `AgentRuntime.snapshot_status()` and command registry.

- Avoid changing provider execution behavior during TUI work.

Storage and Logging

Current persistence:

- JSON state under `runtime/state/`.
- Session/event logs under `runtime/logs/`.
- Obsidian-style local vault under `runtime/obsidian_vault/`.
- Memory store paths controlled by `runtime/tools/memory.py`.

Database state:

- No PostgreSQL integration detected.
- No SQL migration layer detected.

Safety state:

- Execution outputs are logged as replay-only operational events.
- Existing tests verify execution results do not enter evidence memory.

Fixes Applied

1. Added missing dependency:
 - `runtime/requirements.txt`: added `beautifulsoup4>=4.12.0`.
2. Stabilized execution approval:
 - `runtime/tools/executor.py` now requests approval only for actions marked `requires_confirmation` or shell commands classified as `confirmation-required`.
 - Safe filesystem/local actions no longer break noninteractive tests.
 - Risky install commands still prompt and can be rejected.
3. Restored bounded Aureon offline diagnostic behavior:
 - `runtime/providers/aureon_provider.py` now allows only a simple diagnostic greeting when no live `AUREON_API_BASE_URL` exists.
 - Operational planning still fails without a configured backend.
4. Fixed provider alias normalization:
 - `openrouter` alias maps to `openrouter/free` as expected by existing tests.
 - Removed-provider names can still normalize as strings, but provider construction remains blocked elsewhere.
5. Stabilized plain help:
 - `runtime/main.py` now handles `help` and `?` locally through `/help`.
6. Preserved single-action model compatibility:
 - Planner path now accepts a direct action payload when no `plan` key exists.
 - After successful non-final planned actions, runtime reports partial completion instead of silently stopping.
7. Made URL bootstrap noninteractive:

- Browser bootstrap actions use `require_approval=False` for local pre-model context setup.
8. Added browser fallback mode:
- `runtime/tools/browser_tools.py` now falls back to a deterministic local file-mode browser when Playwright/Chromium cannot launch in the current sandbox.
 - This preserves tests and local file inspection behavior without claiming full browser parity.

Verification Results

Commands run:

```
runtime/.venv/bin/python -m compileall -q runtime tests
PYTHONPATH=runtime runtime/.venv/bin/python -m unittest discover -s tests -v
```

Results:

Check	Result
Python compileall	PASS
Import check: <code>bs4</code>	PASS
Runtime smoke import	PASS
Webapp status import	PASS
Retrieval facade smoke	PASS
Shell script syntax check	PASS
Full unittest suite	PASS, 112 tests

Full suite:

Ran 112 tests in 7.546s - OK

Remaining Blockers

1. No real FastAPI backend layer exists.
2. No websocket server is wired.
3. No Textual/Rich TUI exists yet.
4. No PostgreSQL persistence layer exists.
5. Docker setup is not present in the active repo.
6. Runtime package structure is still `PYTHONPATH=runtime` based.
7. Playwright may fail in restricted sandboxes; fallback mode covers local deterministic tests but not full web automation.
8. Provider availability depends on external API keys for real model planning.

Runtime Readiness

Current readiness:

Layer	Readiness
Deterministic Linux retrieval	strong
Provenance attachment	strong for retrieval
Memory isolation smoke coverage	present
Local command registry	usable
Provider switching	usable
Browser tooling	usable with fallback caveat
Local web status API	usable
Terminal prototype foundation	ready for first TUI wrapper
Production API backend	not ready

Prototype Readiness Estimate

The repository is stable enough to start the first terminal prototype phase.

Estimated readiness for first working terminal prototype:

4-8 weeks remains plausible, assuming the next work focuses on:

- CLI/TUI shell over existing runtime;
- stable config loading;
- status/dashboard views;
- provider switching UI;
- log streaming;
- safe shutdown;
- no architecture expansion.

Recommended Architecture Freeze Boundaries

Do not change during the next phase:

- deterministic retrieval facade;
- canonical/candidate knowledge indexes;
- memory authority boundaries;
- provider adapter contract;
- execution approval semantics;
- Phase 0A-0C retrieval guard semantics;
- candidate triage safety rules.

Allowed next-phase changes:

- add a TUI wrapper;
- add status/log views;
- add read-only config display;
- add launch script;
- add smoke tests for terminal UX;
- add docs for developer workflow.

Suggested 7-Day Roadmap

Day 1:

- Add `scripts/dev_check.sh` for compile, import, and unittest checks.
- Add a README section for local runtime startup.

Day 2:

- Create a minimal `tui/` prototype wrapper that imports `AgentRuntime` without moving runtime code.

Day 3:

- Add TUI status dashboard: model, provider status, retrieval flag, session log path, current cwd.

Day 4:

- Add log tail panel for session logs and replay-only command logs.

Day 5:

- Add provider switch controls mapped to existing `/model` behavior.

Day 6:

- Add safe shutdown flow and config reload display.

Day 7:

- Run full test suite, write prototype readiness report, decide whether FastAPI/websocket work is actually necessary.